



Chapter 2: Elementary Programming

The Basics of Java



Objectives

- ✦ To write Java programs to perform simple computations (§2.2).
- ✦ To obtain input from the console using the **Scanner** class (§2.3).
- ✦ To use identifiers to name variables, constants, methods, and classes (§2.4).
- ✦ To use variables to store data (§§2.5–2.6).
- ✦ To program with assignment statements and assignment expressions (§2.6).
- ✦ To use constants to store permanent data (§2.7).
- ✦ To name classes, methods, variables, and constants by following their naming conventions (§2.8).
- ✦ To explore Java numeric primitive data types: **byte**, **short**, **int**, **long**, **float**, and **double** (§2.9.1).
- ✦ To read a **byte**, **short**, **int**, **long**, **float**, or **double** value from the keyboard (§2.9.2).
- ✦ To perform operations using operators **+**, **-**, *****, **/**, and **%** (§2.9.3).
- ✦ To perform exponent operations using **Math.pow(a, b)** (§2.9.4).
- ✦ To write integer literals, floating-point literals, and literals in scientific notation (§2.10).
- ✦ To write and evaluate numeric expressions (§2.11).
- ✦ To obtain the current system time using **System.currentTimeMillis()** (§2.12).
- ✦ To use augmented assignment operators (§2.13).
- ✦ To distinguish between postincrement and preincrement and between postdecrement and predecrement (§2.14).
- ✦ To cast the value of one type to another type (§2.15).
- ✦ To describe the software development process and apply it to develop the loan payment program (§2.16).
- ✦ To write a program that converts a large amount of money into smaller units (§2.17).
- ✦ To avoid common errors and pitfalls in elementary programming (§2.18).



Identifiers

- ◆ What is an identifier?
- ◆ Identifiers are the names that identify elements of your program, such as classes, methods, and variables.
 - An identifier is a sequence of characters that consist of letters, digits, underscores (`_`), and dollar signs (`$`).
 - An identifier must start with a letter, an underscore (`_`), or a dollar sign (`$`). It cannot start with a digit.
 - An identifier cannot be a reserved word. (See Appendix A, “Java Keywords,” for a list of reserved words).
 - An identifier cannot be `true`, `false`, or `null`.
 - An identifier can be of any length.



Identifiers

- ◆ Examples of legal identifiers:
 - `area`, `radius`, `ComputeArea`, `$2`, `average`
- ◆ Examples of illegal identifiers:
 - `2A` and `d+4`
 - ◆ These two do not follow the rules
 - ◆ Java will report that you have a syntax error!
- ◆ Note: Java is case sensitive
 - `area`, `Area`, and `AREA` all are different identifiers



Java Numerical Data Types

- ◆ Every data type has a range of possible values that it can have/hold
- ◆ Whenever you make a variable or constant, the compiler will allocate (create) memory based on the data type requested
- ◆ Java provides eight primitive data types
- ◆ The following table lists the six numeric data types and their ranges and storage sizes



Java Numerical Data Types

♦ Example:

- The largest value you can save into an integer data type is 2,147,483,647.
 - ♦ This is just over 2 billion
- So what if you try to store a value larger than this into an integer data type?

```
int x = 2147483648;
```

- Answer: you will get an error.
- This will not work.
- Solution: use a different data type
 - ♦ double, float, long



Numerical Data Types

- ♦ Java uses four data types for integers:
 - byte, short, int, long
- ♦ Which should you use?
 - Choose the type that is most appropriate for your variable.
 - Long is usually unnecessary for most int types
 - ♦ It is larger than needed.
- ♦ Normal usage:
 - `int` is normally used for integers
 - `double` is normally used for real numbers



Java Numerical Data Types

Name	Range	Storage Size
byte	-2^7 to $2^7 - 1$ (-128 to 127)	8-bit signed
short	-2^{15} to $2^{15} - 1$ (-32768 to 32767)	16-bit signed
int	-2^{31} to $2^{31} - 1$ (-2147483648 to 2147483647)	32-bit signed
long	-2^{63} to $2^{63} - 1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
double	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754



Number Literals

- ♦ A *literal* is a constant value that appears directly in the program.
- ♦ For example, 34, 1,000,000, and 5.0 are literals in the following statements:

```
int i = 34;
```

```
long x = 1000000;
```

```
double d = 5.0;
```



Integer Literals

- ♦ An integer literal can be assigned to an integer variable as long as it can fit into the variable.
- ♦ A compilation error would occur if the literal were too large for the variable to hold.
- ♦ For example, the statement

```
byte b = 1000;
```

 - would cause a compilation error
 - 1000 cannot be stored in a variable of the byte type.
 - The range for byte is -128 to 127
 - ♦ Anything smaller or larger will result in an error!



Integer Literals

- ♦ An integer literal is assumed to be of the `int` type
 - The range for `int` is between -2^{31} (-2147483648) to $2^{31}-1$ (2147483647).
 - If you want an `int`, but you need to store a larger number, then you should use the type `long`
 - To denote an integer literal of the `long` type, append it with the letter `L` or `l`.
 - ♦ `L` is preferred because `l` (lowercase `L`) can easily be confused with `1` (the digit one).



Floating-Point Literals

- ◆ Floating-point literals are written with a decimal point.
 - By default, a floating-point literal is treated as a double type value.
 - For example, 5.0 is considered a double value, not a float value.
 - You can make a number a float by appending the letter f or F, and make a number a double by appending the letter d or D.
 - ◆ For example, you can use 100.2f or 100.2F for a float number, and 100.2d or 100.2D for a double number.

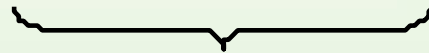


double vs. float

The double type values are more accurate than the float type values. For example,

```
System.out.println("1.0 / 3.0 is " + 1.0 / 3.0);
```

displays `1.0 / 3.0 is 0.3333333333333333`



16 digits

```
System.out.println("1.0F / 3.0F is " + 1.0F / 3.0F);
```

displays `1.0F / 3.0F is 0.33333334`



7 digits



Java Variables

- ◆ Variables are used to represent values that may be changed in the program.
- ◆ They are called variables because their values can be changed!



Declaring Variables

♦ Syntax for declaring a variable:

```
datatype variableName;
```

♦ Examples:

```
int x;           // Declare x to be an
                  // integer variable;

double radius;  // Declare radius to
                  // be a double variable;

char a;          // Declare a to be a
                  // character variable;
```



Declaring Variables

- ♦ If variables are of the same data type, they can be declared together:

```
datatype var1, var2, var3, ..., varn;
```

- ♦ Example:

```
int i, j, k;
```

- ♦ Variables often have initial values
- ♦ You can declare and initialize in one step:

```
int count = 1;
```

```
double pi = 3.14159;
```



Declaring Variables

- ◆ You can also use shorthand form to declare and initialize variables of the same type together:

- ◆ Example:

```
int i = 62, j = 78, k = 35;
```

- ◆ Tip:

- A variable must be declared before it can be assigned a value, and a value must be assigned to the variable before it can be used.
- Try to declare and initialize in one step.
 - ◆ This makes program easier to read and helps to avoid errors



Assignment Statements

- ◆ After a variable has been declared, we can give that variable a value.
- ◆ This is called “**assigning a value**” to the variable.
- ◆ In Java, we do this with the *assignment statement*.
 - The equal sign (=) is used as the *assignment operator*.
 - The syntax for assignment statement is as follows:

```
variable = value;
```


or

```
variable = expression;
```



Assignment Statements

- ◆ Sometimes we assign an exact values into variables:

- Examples:

```
int y = 1;           // assign 1 to y
double w = 3.0;       // assign 3.0 to w
```

- ◆ Other times we assign the value of an expression into the variable:

- Examples:

```
int x = 5 * (3 / 2);
double area = radius * radius * 3.14159;
```



Assignment Statements

- ◆ If a value is assigned to multiple variables, you can use this syntax:

```
i = j = k = 5;
```

- ◆ This is equivalent to:

```
k = 5;
```

```
j = k;
```

```
i = j;
```



Assignment Statements

- ◆ You can also use the same variable on both sides of the assignment statement

- ◆ Example:

$$x = x + 1;$$

- First, the right side of the assignment is calculated.
- Then, the new value is assigned into the variable on the left (**x**).
 - ◆ So if the value of x was 7 before the statement is executed, then x will become 8 after the statement is executed.

- ◆ See sample program: `VariablePractice.java`



Assignment Statements

♦ Note: in an assignment statement, the data type of the variable on the left must be compatible with the data type of the value on the right.

♦ Example:

```
int x = 1.0;
```

- This would be illegal!
- The data type of `x` is an `int`.
- You cannot assign a `double` value (`1.0`) into an `int` variable unless you use type casting.
 - ♦ Type casting is coming later...



Writing a Simple Program

- ♦ Write a program that will calculate the area of a circle.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Writing a Simple Program

- ♦ Write a program that will calculate the area of a circle.
- ♦ **Step 1: Design your algorithm**
 1. Get the radius of the circle.
 2. Compute the area using the following formula:
 - ♦ $\text{area} = \text{radius} \times \text{radius} \times \pi$
 3. Display the result



Writing a Simple Program

- ♦ Write a program that will calculate the area of a circle.
- ♦ **Step 2: Implementation (code the algorithm)**
 - In order to store the radius, the program must declare a symbol called a **variable**.
 - A variable represents a value stored in the computer's memory
 - You should choose good names for variables
 - ♦ Do not choose “x” or “y”...these have no meaning
 - ♦ Choose names with meaning...“area” or “radius”



Writing a Simple Program

- ♦ Write a program that will calculate the area of a circle.
- ♦ **Step 2: Implementation (code the algorithm)**
 - What value do you want to store in *radius*?
 - What about *area*?
 - ♦ Integer? Real number? Something else maybe?
 - The variable's **data type** is the kind of data that you can store in that particular variable.
 - So when you **declare** (create) a variable, you must state its data type and the variable name.



Writing a Simple Program

- ♦ Write a program that will calculate the area of a circle.
- ♦ **Step 2: Implementation (code the algorithm)**
 - Java provides simple data types for integers, real numbers, characters, and Boolean types.
 - These basic data types are known as **primitives**.
 - Here are two example primitive data types:
 - ♦ `int` : used to store an integer
 - ♦ `double` : used to store a real number



Writing a Simple Program

- ✦ Write a program that will calculate the area of a circle.
- ✦ **Step 2: Implementation (code the algorithm)**

```
public class ComputeArea {  
    public static void main(String[] args) {  
        double radius, area;  
        // Step 1: get radius  
  
        // Step 2: calculate area  
  
        // Step 3: display the result  
  
    }  
}
```



Writing a Simple Program

- ♦ Write a program that will calculate the area of a circle.
- ♦ **Step 2: Implementation (code the algorithm)**
 - Now, we set a value for radius.
 - ♦ Later we will learn how to ask the user to input the value for radius!
 - We then perform the calculation to get the area.
 - And we print/display the result.



Writing a Simple Program

- ✦ Write a program that will calculate the area of a circle.
- ✦ **Step 2: Implementation (code the algorithm)**

```
public class ComputeArea {  
    public static void main(String[] args) {  
        double radius, area;  
        // Step 1: set radius  
        radius = 20;  
        // Step 2: calculate area  
        area = radius * radius * 3.14159;  
  
        // Step 3: display the result  
        System.out.println("The area for the circle of radius " +  
            radius + " is " + area + ".");  
    }  
}
```

Trace a Program Execution

```
1 public class ComputeArea {  
2     public static void main(String[] args) {  
3         double radius;  
4         double area;  
5  
6         // Assign a radius  
7         radius = 20;  
8  
9         // Compute area  
10        area = radius * radius * 3.14159;  
11  
12        // Display results  
13        System.out.println("The area for the circle of radius " +  
14            radius + " is " + area);  
15    }  
16 }
```

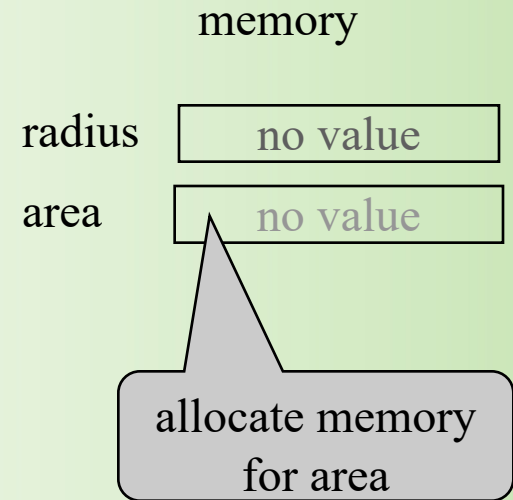
allocate memory
for radius

radius

no value

Trace a Program Execution

```
1 public class ComputeArea {  
2     public static void main(String[] args) {  
3         double radius;  
4         double area;  
5  
6         // Assign a radius  
7         radius = 20;  
8  
9         // Compute area  
10        area = radius * radius * 3.14159;  
11  
12        // Display results  
13        System.out.println("The area for the circle of radius " +  
14        radius + " is " + area);  
15    }  
16 }
```



Trace a Program Execution

```
1 public class ComputeArea {  
2     public static void main(String[] args) {  
3         double radius;  
4         double area;  
5  
6         // Assign a radius  
7         radius = 20;  
8  
9         // Compute area  
10        area = radius * radius * 3.14159;  
11  
12        // Display results  
13        System.out.println("The area for the circle of radius " +  
14            radius + " is " + area);  
15    }  
16 }
```

radius

area

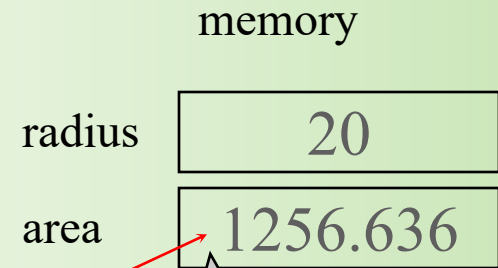
assign 20 to radius

20

no value

Trace a Program Execution

```
1 public class ComputeArea {  
2     public static void main(String[] args) {  
3         double radius;  
4         double area;  
5  
6         // Assign a radius  
7         radius = 20;  
8  
9         // Compute area  
10        area = radius * radius * 3.14159;  
11  
12        // Display results  
13        System.out.println("The area for the circle of radius " +  
14            radius + " is " + area);  
15    }  
16 }
```



compute area and assign it to variable area

Trace a Program Execution

```
1 public class ComputeArea {  
2     public static void main(String[] args) {  
3         double radius;  
4         double area;  
5  
6         // Assign a radius  
7         radius = 20;  
8  
9         // Compute area  
10        area = radius * radius * 3.14159;  
11  
12        // Display results  
13        System.out.println("The area for the circle of radius " +  
14        radius + " is " + area);  
15    }  
16 }
```

memory

radius

20

area

1256.636

print a message to the
console





Writing a Simple Program

♦ Discussion:

- Variables such as radius and area refer to memory locations
- Each variable has a name, a type, a size, and value
- Line 3 says that radius can store a double value.
 - ♦ But the value is not defined until you assign a value.
- Line 7 assigns the value 20 into radius.
- Similarly, line 4 declares the variable area.
- Line 10 then assigns a value into area.



Writing a Simple Program

♦ Discussion:

- The following table shows the value in memory for the variables area and radius as the program is executed.

line#	radius	area
3	no value	
4		no value
7	20	
10		1256.636

- This method of reviewing a program is called “*tracing a program*”.
- Helps you to understand how programs work.



Writing a Simple Program

♦ Discussion:

- The plus sign (+) has two meanings in Java:
 - ♦ Addition
 - ♦ Concatenation (combining two strings together)
- The plus sign (+) in lines 13-14 is called a string concatenation operator.
 - ♦ It combines two strings into one string.
 - ♦ If a string is concatenated with a number, the number is converted into a string and then concatenated with the other string.

♦ See sample program: `StringPractice.java`



Writing a Simple Program

♦ Discussion:

- In Java, a string cannot be written across multiple lines in the source code.
- The following statement would result in a **compile error**:

```
System.out.println("Introduction to Java Programming, by  
Y. Daniel Liang");
```

- To fix the error, break the string into separate substrings and use concatenation (+) to combine:

```
System.out.println("Introduction to Java Programming, " +  
" by Y. Daniel Liang");
```



Reading Input from the Console

- ◆ In the last example, the radius was fixed.
- ◆ Your program can be better, and more interactive, by letting the user enter the radius.
- ◆ Java uses the **Scanner** class for console input
 - `System.out` refers to the standard output device
 - ◆ By default, standard output is the monitor
 - `System.in` refers to the standard input device
 - ◆ By default, standard input is the keyboard



Reading Input from the Console

- ♦ Java uses the **Scanner** class for console input
 - So how do you read from the keyboard?
 - You make a Scanner object!
 - And you tell this Scanner object to read from System.in (they keyboard)
 - Here is the code:

```
Scanner input = new Scanner(System.in);
```



Reading Input from the Console

```
Scanner input = new Scanner(System.in);
```

- `new Scanner(System.in)` creates an object of the `Scanner` type.
- `Scanner input` says that `input` is a variable whose type is `Scanner`.
- The whole line creates a `Scanner` object and then saves the reference of this object into the variable called `input`.
- Now, we can use this new `Scanner` object.
 - ◆ Specifically, this `Scanner` object has helpful methods that allow us to read/scan data from the user.



Reading Numbers from the Keyboard

```
Scanner input = new Scanner(System.in) ;  
int value = input.nextInt() ;
```

Method	Description
<code>nextByte()</code>	reads an integer of the byte type.
<code>nextShort()</code>	reads an integer of the short type.
<code>nextInt()</code>	reads an integer of the int type.
<code>nextLong()</code>	reads an integer of the long type.
<code>nextFloat()</code>	reads a number of the float type.
<code>nextDouble()</code>	reads a number of the double type.

```
double grade = input.nextDouble() ;
```



Reading Input from the Console

♦ Methods of Scanner object:

- We can invoke a method on the Scanner object.
- What does this mean?
- It means we are asking the Scanner object to perform a specific task.
- Example:
 - ♦ We can invoke the `nextDouble()` method
 - ♦ This allows us to read a double value from the input

```
double radius = input.nextDouble();
```



Reading Input from the Console

♦ Summary:

- Create a Scanner object:

```
Scanner input = new Scanner(System.in);
```

- Use the method `nextDouble()` to read a double value from the user/keyboard:

```
double radius = input.nextDouble();
```

- Now, let us revisit the Compute Area of a Circle
- We will get the radius from the user...



Program 2: Compute Area with Console/User Input

LISTING 2.2 ComputeAreaWithConsoleInput.java

```
1  import java.util.Scanner; // Scanner is in the java.util package      import class
2
3  public class ComputeAreaWithConsoleInput {
4      public static void main(String[] args) {
5          // Create a Scanner object
6          Scanner input = new Scanner(System.in);                        create a Scanner
7
8          // Prompt the user to enter a radius
9          System.out.print("Enter a number for radius: ");
10         double radius = input.nextDouble();                            read a double
11
12         // Compute area
13         double area = radius * radius * 3.14159;
14
15         // Display results
16         System.out.println("The area for the circle of radius " +
17             radius + " is " + area);
18     }
19 }
```

♦ See sample program: AreaCircle.java



Program 2: Compute Area with Console/User Input

♦ Discussion:

- Before you can use the Scanner class, you must **import** it!
- The Scanner class is in the `java.util` package.
- We import this on Line 1 of the program.
- Notice that we do this import before we start coding our actual class.



Program 2: Compute Area with Console/User Input

♦ Discussion:

- There are two types of import statements:
 - ♦ Specific import: specifies a single class that should be imported
 - Example:

```
import java.util.Scanner;
```
 - ♦ Wildcard import: imports all the classes in a package by using the asterisk as the wildcard.
 - Example:

```
import java.util.*;
```
- You can use either methods to import classes.
- The choice is up to you! You are the programmer!



Program 3: Compute Average

- ♦ Write a program to get three values from the user and compute their average.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 3: Compute Average

- ♦ Write a program to get three values from the user and compute their average.
- ♦ **Step 1: Design your algorithm**
 1. Get three numbers from the user.
 - ♦ Use Scanner object
 2. Compute the average of the three numbers:
 - ♦ $\text{average} = (\text{num1} + \text{num2} + \text{num3}) / 3$
 3. Display the result



Program 3: Compute Average

- ✦ Write a program to get three values from the user and compute their average.
- ✦ **Step 2: Implementation (code the algorithm)**

```
import java.util.Scanner;

public class ComputeAverage {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        // Step 1: ask user to enter three values

        // Step 2: calculate average

        // Step 3: display the results

    }
}
```



Program 3: Compute Average

LISTING 2.3 ComputeAverage.java

```
1  import java.util.Scanner; // Scanner is in the java.util package
2
3  public class ComputeAverage {
4      public static void main(String[] args) {
5          // Create a Scanner object
6          Scanner input = new Scanner(System.in);
7
8          // Prompt the user to enter three numbers
9          System.out.print("Enter three numbers: ");
10         double number1 = input.nextDouble();
11         double number2 = input.nextDouble();
12         double number3 = input.nextDouble();
13
14         // Compute average
15         double average = (number1 + number2 + number3) / 3;
16
17         // Display results
18         System.out.println("The average of " + number1 + " " + number2
19             + " " + number3 + " is " + average);
20     }
21 }
```

♦ See sample program: `AverageFourNumbers.java`

Program 3: Compute Average

♦ Example runs of the program:

Enter three numbers: 1 2 3 ↵ Enter
The average of 1.0 2.0 3.0 is 2.0



enter input in one line

Enter three numbers: 10.5 ↵ Enter
11 ↵ Enter
11.5 ↵ Enter
The average of 10.5 11.0 11.5 is 11.0



enter input in multiple lines



Named Constants

♦ A named constant is an identifier that represents a permanent value.

- The value of a variable can change during execution of a program.
- However, a *named constant*, or simply *constant*, represents a permanent data that **never** changes.
- Here is the syntax:

```
final datatype CONSTANTNAME = value;
```

- Example:

```
final double PI = 3.14159;
```

```
final int SIZE = 15;
```

Program 4: Compute Area with a Constant



LISTING 2.4 ComputeAreaWithConstant.java

```
1  import java.util.Scanner; // Scanner is in the java.util package
2
3  public class ComputeAreaWithConstant {
4      public static void main(String[] args) {
5          final double PI = 3.14159; // Declare a constant
6
7          // Create a Scanner object
8          Scanner input = new Scanner(System.in);
9
10         // Prompt the user to enter a radius
11         System.out.print("Enter a number for radius: ");
12         double radius = input.nextDouble();
13
14         // Compute area
15         double area = radius * radius * PI;
16
17         // Display result
18         System.out.println("The area for the circle of radius " +
19             radius + " is " + area);
20     }
21 }
```



Named Constants

- ♦ So what are the benefits of using constants?
 1. You don't have to repeatedly type the same value if it is used multiple times
 2. If you have to change the value, you only need to change it in a single location in the code
 - ♦ Instead of changing it at all locations
 3. A descriptive name for a constant makes the program easier to read



Naming Conventions

- ♦ Choose meaningful and descriptive names.
 - Do not use abbreviations
- ♦ Variables and method names:
 - Use lowercase. If the name consists of several words, concatenate all in one, use lowercase for the first word, and capitalize the first letter of each subsequent word in the name.
 - For example, the variables `radius` and `area`, and the method `computeArea`.



Naming Conventions

♦ Class names:

- Capitalize the first letter of each word in the name.
- For example, the class name `ComputeArea`.

♦ Constants:

- Capitalize all letters in constants, and use underscores to connect words.
- For example, the constants `PI` and `MAX_VALUE`

♦ Do you have to follow these rules?

- No. But it makes your program **MUCH** easier to read!!!



Numeric Operators

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder	20 % 3	2



Integer Division

- ♦ “Normal” division: $7 / 2 = 3.5$
- ♦ In Computer Science, when we say division, majority of the time, we mean integer division.
 - When both operands of a division are integers, we will use integer division.
- ♦ What is integer division?
 - Easiest to explain with examples:

♦ $5 / 2 = 2$	$12 / 5 = 2$
♦ $7 / 2 = 3$	$15 / 4 = 3$
♦ $15 / 2 = 7$	$33 / 8 = 4$



Remainder Operator

- ♦ The **%** operator is known as the **remainder** operator, or also as the **modulo** operator
 - This operator will give the remainder after division
 - Examples:
 - ♦ $7 \% 3 = 1$
 - ♦ $12 \% 4 = 0$
 - ♦ $20 \% 13 = 7$
 - ♦ $3 \% 7 = 3$
 - ♦ $26 \% 8 = 2$

$\begin{array}{r} 2 \\ 3 \overline{) 7} \\ \underline{6} \\ 1 \end{array}$	$\begin{array}{r} 0 \\ 7 \overline{) 3} \\ \underline{0} \\ 3 \end{array}$	$\begin{array}{r} 3 \\ 4 \overline{) 12} \\ \underline{12} \\ 0 \end{array}$	$\begin{array}{r} 3 \\ 8 \overline{) 26} \\ \underline{24} \\ 2 \end{array}$	Divisor $\longrightarrow$	$\begin{array}{r} 1 \\ 13 \overline{) 20} \\ \underline{13} \\ 7 \end{array}$	$\longleftarrow$ Quotient	$\longleftarrow$ Dividend	$\longleftarrow$ Remainder
--	--	--	--	---------------------------	---	---------------------------	---------------------------	----------------------------



Remainder Operator

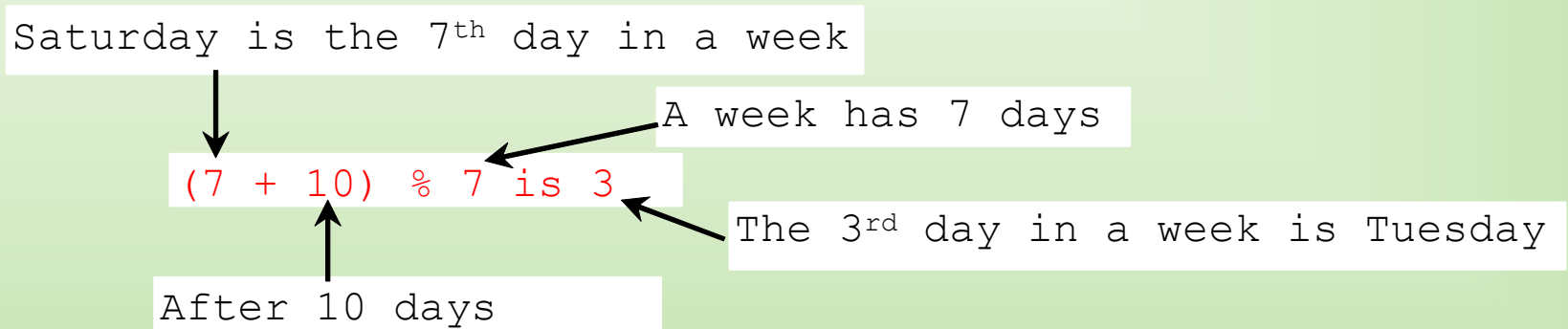
- ◆ Remainder is very useful in programming.
- ◆ For example, an even number % 2 is always 0
- ◆ An odd number % 2 is always 1
- ◆ So you can use this property to determine whether a number is even or odd.
- ◆ You can also mod by other values to achieve valuable results.



Remainder Operator

♦ Example:

- If today is Saturday, it will be Saturday again in 7 days. Suppose you and your friends will meet in 10 days. What day is it in 10 days?
- Let us assume Sunday is the 1st day of the week.
- We can find that in 10 days, the day will be Tuesday by using the following equation:





Program 5: Convert Time

- ♦ Write a program to get an amount of time from the user in seconds. Then your program should convert this time into minutes and the remaining seconds.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 5: Convert Time

♦ Step 1: Problem-solving Phase

- If you are given seconds, how do you then calculate the minutes and remaining seconds?

♦ Example:

- Given 624 seconds, how do we calculate the minutes?
- We divide by 60!
 - We see how many complete 60s are in 624.
 - Answer: 10 of them. $10 \times 60 = 600$.
- So in 624 seconds, there are a full 10 minutes.
- After we remove those 10 minutes, how many seconds are remaining?
 - $624 - (10 \times 60) = 24$ seconds remaining
 - We can use mod! $624 \% 60 = 24$ seconds remaining.



Program 5: Convert Time

♦ Step 1: Design your algorithm

1. Get amount of seconds from the user.
 - ♦ Use Scanner object
 - ♦ Save as an int
2. Compute the minutes and seconds remaining:
 - ♦ From these seconds, determine the number of minutes
 - ♦ Example:
 - 150 seconds $\Rightarrow$ 2 minutes and 30 seconds
 - 315 seconds $\Rightarrow$ 5 minutes and 15 seconds
 - $315 / 60 = 5$ and $315 \% 60 = 15$
3. Display the result



Program 5: Convert Time

♦ Step 2: Implementation

LISTING 2.5 DisplayTime.java

```
1  import java.util.Scanner;
2
3  public class DisplayTime {
4      public static void main(String[] args) {
5          Scanner input = new Scanner(System.in);
6          // Prompt the user for input
7          System.out.print("Enter an integer for seconds: ");
8          int seconds = input.nextInt();
9
10         int minutes = seconds / 60; // Find minutes in seconds
11         int remainingSeconds = seconds % 60; // Seconds remaining
12         System.out.println(seconds + " seconds is " + minutes +
13             " minutes and " + remainingSeconds + " seconds");
14     }
15 }
```

Program 5: Convert Time

Enter an integer for seconds: 500

500 seconds is 8 minutes and 20 seconds



line#	seconds	minutes	remainingSeconds
8	500		
10		8	
11			20



♦ See sample program: TimeCalculation.java



NOTE

- ✦ Calculations involving floating-point numbers are approximated because these numbers are not stored with complete accuracy.

- For example:

- `System.out.println(1.0-0.1-0.1-0.1-0.1-0.1);`

- ✦ displays 0.5000000000000000001, not 0.5, and

- `System.out.println(1.0 - 0.9);`

- ✦ displays 0.09999999999999999998, not 0.1.

- Integers are stored precisely.
- Therefore, calculations with integers yield a precise integer result.



Exponent Operations

```
System.out.println(Math.pow(2, 3));
```

```
// Displays 8.0
```

```
System.out.println(Math.pow(2, 5));
```

```
// Displays 32.0
```

```
System.out.println(Math.pow(4, 0.5));
```

```
// Displays 2.0
```

```
System.out.println(Math.pow(2.5, 2));
```

```
// Displays 6.25
```

```
System.out.println(Math.pow(2.5, -2));
```

```
// Displays 0.16
```



Arithmetic Expressions

- ♦ Java expressions are written the same way as normal arithmetic expressions.
- ♦ Example:

$$\frac{3 + 4x}{5} - \frac{10(y - 5)(a + b + c)}{x} + 9\left(\frac{4}{x} + \frac{9 + x}{y}\right)$$

- ♦ is translated into

$$(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x + 9 * (4 / x + (9 + x) / y)$$



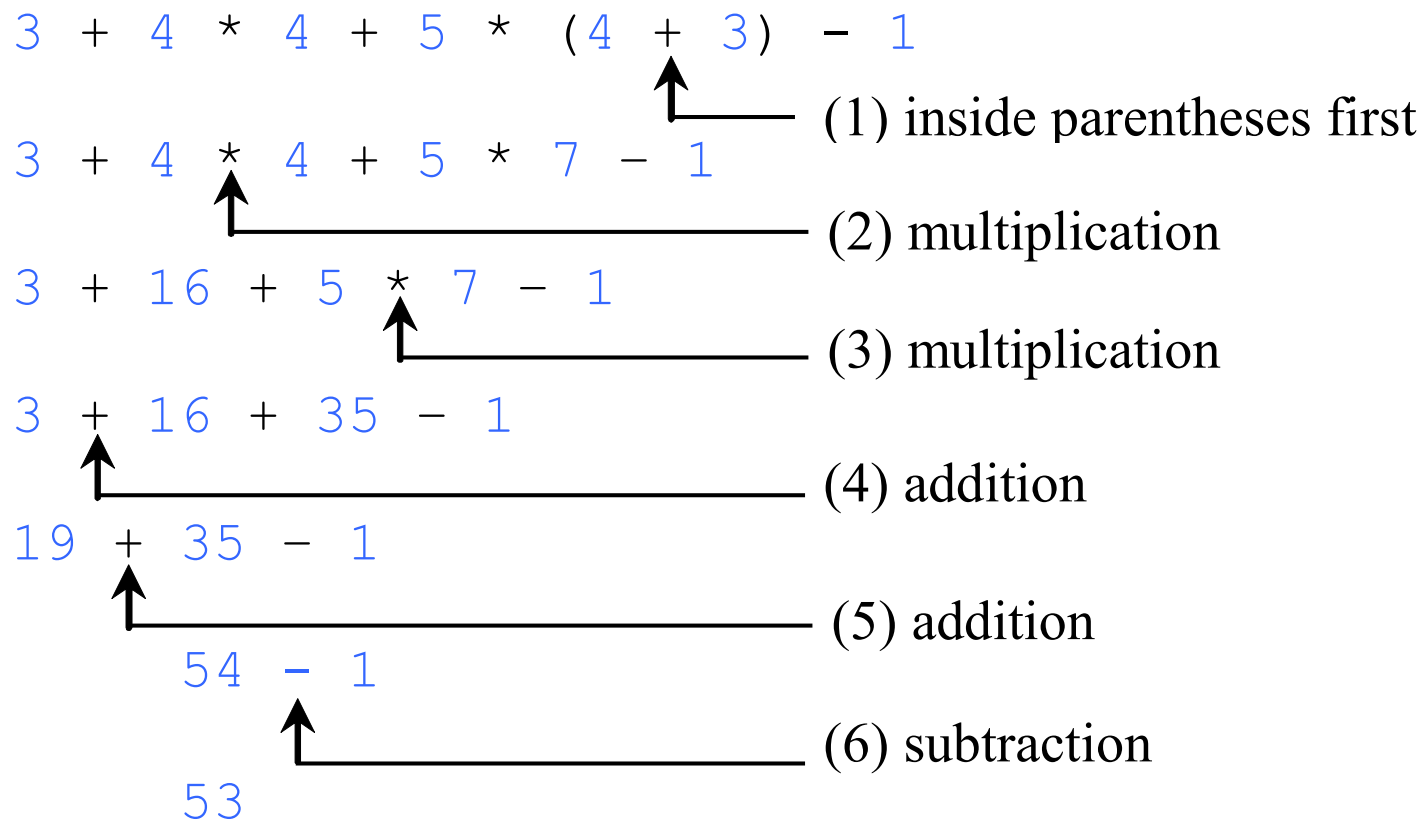
How to Evaluate an Expression

- ♦ Summary: you can safely apply the arithmetic rule for evaluating a Java expression
 - Operators inside parenthesis are evaluated first
 - ♦ Parenthesis can be nested
 - ♦ Expression in inner parenthesis is evaluated first
 - Use normal operator precedence
 - ♦ Multiplication, division, and remainder are done first
 - if an expression has several multiplication, division, and remainder operators, you evaluate them from left to right
 - ♦ Addition and subtraction are done last
 - and again, if an expression has several addition and subtraction operators, you evaluate them from left to right



How to Evaluate an Expression

♦ Example of how an expression is evaluated:





Program 6: Convert Temperature

- ♦ Write a program that converts a temperature in Fahrenheit into Celsius.
 - You will need the following formula:

$$celsius = (\frac{5}{9})(fahrenheit - 32)$$

- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 6: Convert Temperature

♦ Step 1: Design your algorithm

1. Get temperature in Fahrenheit from the user.

- ♦ Use Scanner object
- ♦ Save temperature as an int

2. Compute the temperature into Celsius:

- ♦ Use formula: $celsius = (\frac{5}{9})(fahrenheit - 32)$
- ♦ But write the formula in Java:

```
celsius = (5.0 / 9) * (fahrenheit - 32);
```

3. Display the result



Program 6: Convert Temperature

♦ Step 2: Implementation

LISTING 2.6 FahrenheitToCelsius.java

```
1  import java.util.Scanner;
2
3  public class FahrenheitToCelsius {
4      public static void main(String[] args) {
5          Scanner input = new Scanner(System.in);
6
7          System.out.print("Enter a degree in Fahrenheit: ");
8          double fahrenheit = input.nextDouble();
9
10         // Convert Fahrenheit to Celsius
11         double celsius = (5.0 / 9) * (fahrenheit - 32);
12         System.out.println("Fahrenheit " + fahrenheit + " is " +
13             celsius + " in Celsius");
14     }
15 }
```



Program 6: Convert Temperature

♦ Discussion: be careful when dividing integers

- Notice the formula has 5 divided by 9

$$celsius = (\frac{5}{9})(fahrenheit - 32)$$

- What happens if we write this formula as:

```
celsius = (5 / 9) * (fahrenheit - 32);
```

- (5 / 9) evaluates to zero!

- ♦ Integer division!

- So we use (5.0 / 9) instead, which gives a number

♦ See sample program: TempConvert.java



Augmented Assignment Operators

- ♦ Very often, we use the current value of a variable, we modify it, and then save it back to the same variable.

- Example:

```
count = count + 1;
```

- Java allows you to combine this addition and assignment into one operator, which is called the augmented assignment operator.

- Example:

```
count += 1;
```



Augmented Assignment Operators

<i>Operator</i>	<i>Name</i>	<i>Example</i>	<i>Equivalent</i>
<code>+=</code>	Addition assignment	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	Subtraction assignment	<code>i -= 8</code>	<code>i = i - 8</code>
<code>*=</code>	Multiplication assignment	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	Division assignment	<code>i /= 8</code>	<code>i = i / 8</code>
<code>%=</code>	Remainder assignment	<code>i %= 8</code>	<code>i = i % 8</code>



Augmented Assignment Operators

- ♦ The augmented assignment operator is performed last after all the other operators in the expression are evaluate

- Example:

`x /= 4 + 5.5 * 1.5;`

is same as

`x = x / (4 + 5.5 * 1.5);`

- Caution: there are no spaces in the augmented operators

- ♦ For example, `+ =` should be `+=` (with no space)



Increment and Decrement Operators

- ♦ Another common expression is to simply increment (increase) a variable by one
 - Such as $x = x + 1$;
 - Because this is so common, Java gives you special increment and decrement operators
 - ♦ Increment operator: **++**
 - ♦ Decrement operator: **--**
 - Examples:

```
int i = 3, j = 3;
i++;    // i becomes 4
j--;    // j becomes 2
```



Increment and Decrement Operators

♦ More details:

- ♦ `i++` is pronounced as `i` plus plus
- ♦ `i--` is pronounced as `i` minus minus
- These two operators are known as postincrement and postdecrement
- Why?
 - ♦ Because the operators (`++` and `--`) are placed after the variable
- The operators can also be placed before the variable



Increment and Decrement Operators

♦ More details:

- The operators can also be placed before the variable

```
int i = 3, j = 3;
```

```
++i; // i becomes 4
```

```
--j; // j becomes 2
```

- Again, ++i increments i, and --j decrements j
- In this small example, result is the same
 - ♦ Meaning i++ and ++i both increase i from 3 to 4.
 - ♦ And both --j and j-- decrease j from 3 to 2.



Increment and Decrement Operators

♦ More details:

- If the statement is ONLY doing increment or decrement, the effect of `j++` and `++j` is the same.
- However, the effect changes when these operators are used in other types of statements.

♦ Specifically:

- `++i`: the increment is done before evaluating an expression
- `i++`: the increment is done after evaluated an expression
- Study the following table and examine the results...



Increment and Decrement Operators

<i>Operator</i>	<i>Name</i>	<i>Description</i>	<i>Example (assume i = 1)</i>
++var	preincrement	Increment var by 1 , and use the new var value in the statement	int j = ++i; // j is 2, i is 2
var++	postincrement	Increment var by 1 , but use the original var value in the statement	int j = i++; // j is 1, i is 2
--var	predecrement	Decrement var by 1 , and use the new var value in the statement	int j = --i; // j is 0, i is 0
var--	postdecrement	Decrement var by 1 , and use the original var value in the statement	int j = i--; // j is 1, i is 0

Increment and Decrement Operators

♦ Consider the following code:

```
int i = 10;  
int newNum = 10 * i++;  
System.out.print("i is " + i  
    + ", newNum is " + newNum);
```

Same effect as →

```
int newNum = 10 * i;  
i = i + 1;
```

```
i is 11, newNum is 100
```



– Details:

- ♦ Here, we first get the value of i (which is 10) and calculate newNum.
- ♦ Then, we increment i.
 - i is not incremented until AFTER the expression is evaluated.

Increment and Decrement Operators

♦ Consider the following code:

```
int i = 10;  
int newNum = 10 * (++i);  
System.out.print("i is " + i  
    + ", newNum is " + newNum);
```

Same effect as

```
i = i + 1;  
int newNum = 10 * i;
```



```
i is 11, newNum is 110
```

– Details:

- ♦ Here, we first increment i
- ♦ Then, after i is incremented, we calculate newNum



Increment and Decrement Operators

♦ Another example:

- Consider the following code:

```
double x = 1.0;
```

```
double y = 5.0;
```

```
double z = x-- + (++y);
```

- What is the value of x, y, and z after the three lines are executed?

- ♦ x becomes 0.0

- ♦ y becomes 6.0

- ♦ z becomes 7.0



Increment and Decrement Operators

♦ More details:

- Using increment and decrement operators makes expressions short
 - ♦ but it also makes them complex and difficult to read
- Avoid using these operators in expressions that modify multiple variables, or the same variable for multiple times such as this:

```
int k = ++i + i;
```

♦ Is this legal?

- Yes. But it is confusing!
- Message: don't use increment/decrement like this.



Numeric Type Conversions

- ♦ Can you perform binary operations with operands of different types?
 - Meaning, can we add an integer literal with a double literal?
 - Answer: YES.
 - ♦ If you add an integer with a floating-point number, Java automatically converts the int to a floating point value.
 - Example:
 - ♦ $3 * 4.5$ is the same as
 - ♦ $3.0 * 4.5$



Numeric Type Conversions

◆ Details:

- You can always assign a value to a numeric variable just so long as the value is within the limits/range of the data type.

- Example:

- ◆ You can save an int into a double, because the double is much wider (larger) than the int

```
int x = 4;
```

```
double y;
```

```
y = x;
```

- ◆ This is allowed, because x can easily “fit” into y.



Numeric Type Conversions

◆ Details:

- However, you cannot assign a value to a variable of a data type with a smaller range of values.
 - ◆ Unless you use *type casting*
- ***Casting*** is an operation that converts a value of one data type into a value of another data type
 - ◆ Casting a type with a small range to a type with a larger range is known as “widening a type”
 - ◆ Casting a type with a large range to a type from a smaller range is known as “narrowing a type”



Numeric Type Conversions

♦ Casting:

- Java will automatically widen a type, but you must request a narrowing explicitly

- Syntax:

- ♦ specify the target type in parentheses, followed by the variable's name or the value to be cast

- Example:

```
System.out.println((int)1.7);
```

- ♦ 1 gets printed.

- ♦ Why? Because 1.7 was converted into an int.



Numeric Type Conversions

♦ Casting:

– Example:

```
System.out.println( (double) 1 / 2 );
```

♦ 0.5 gets printed.

♦ Why? Because 1 is cast into 1.0. Then 1.0 is divided by 2.

– Example:

```
System.out.println( 1 / 2 );
```

♦ Be careful!

♦ Here, 0 (zero) gets printed. Why? 1 and 2 are both integers and the result should be an integer.



Type Casting

Implicit casting

```
double d = 3; // (type widening)
```

Explicit casting

```
int i = (int)3.0; // (type narrowing)
```

```
int i = (int)3.9; // (Fraction part is truncated)
```

What is wrong? `int x = 5 / 2.0;`

range increases



byte, short, int, long, float, double

♦ See sample program: `JavaBasics.java`



Conversion Rules

When performing a binary operation involving two operands of different types, Java automatically converts the operand based on the following rules:

1. If one of the operands is double, the other is converted into double.
2. Otherwise, if one of the operands is float, the other is converted into float.
3. Otherwise, if one of the operands is long, the other is converted into long.
4. Otherwise, both operands are converted into int.



Program 7: Sales Tax

- ♦ Write a program that reads a purchase amount from the user, calculates the sales tax, and then displays the result.
 - But we want to print the tax with only two decimal places
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 7: Sales Tax

♦ Step 1: Design your algorithm

1. Get purchase amount from user.
 - ♦ Use Scanner object
 - ♦ Save `purchaseAmount` as a double
2. Compute the tax:
 - ♦ Use a simple tax rate (6%)
 - ♦ Use the formula:
 - $\text{tax} = \text{purchaseAmount} * 0.06$
3. Display the result
 - ♦ Print the result, but show only two decimal places



Program 7: Sales Tax

♦ Example:

- If `purchaseAmount` is entered as 197.55
- Then the sales tax is evaluated as $197.55 * 0.06$
 - ♦ This equals **11.853**
- So how do we display this with only two decimals?
 1. Multiply 11.853 by 100
 - $11.853 * 100 = \mathbf{1185.3}$
 2. Cast the result as an integer
 - Now 1185.3 becomes **1185** (the .3 is removed)
 3. Now, divide by 100.0, which is a double
 - And we get **11.85**



Program 7: Sales Tax

♦ Step 2: Implementation

LISTING 2.8 SalesTax.java

```
1  import java.util.Scanner;
2
3  public class SalesTax {
4      public static void main(String[] args) {
5          Scanner input = new Scanner(System.in);
6
7          System.out.print("Enter purchase amount: ");
8          double purchaseAmount = input.nextDouble();
9
10         double tax = purchaseAmount * 0.06;
11         System.out.println("Sales tax is $" + (int)(tax * 100) / 100.0);
12     }
13 }
```

♦ See sample program: SalesTax.java



Casting in an Augmented Expression

- ◆ In Java, an **augmented expression** of the form **x1 op= x2** is implemented as

- **x1 = (T)(x1 op x2)**

Here, OP means operation.

- ◆ where T is the type for x1.

- ◆ Example

```
int sum = 0;
```

```
sum += 4.5; // sum becomes 4 after this statement
```

- This is equivalent to:

- ◆ `sum = (int)(sum + 4.5);`



Program 8: Money Units

- ♦ Write a program that asks the user for an amount of money in dollars and cents. Then your program should output a report listing the number of dollars, quarters, dimes, nickels, and pennies (and in that order) in your program.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 8: Money Units

♦ Step 1: Problem-solving Phase

- A reminder about U.S. monetary units:
 - ♦ 1 dollar = 100 cents (or pennies)
 - ♦ 1 quarter = 25 cents
 - ♦ 1 dime = 10 cents
 - ♦ 1 nickel = 5 cents
- So if you need to give someone 42 cents in change, you should give:
 - ♦ One quarter, one dime, one nickel, and two pennies



Program 8: Money Units

♦ Step 1: Problem-solving Phase

- First step: UNDERSTAND the problem!
- So let us look at an example run:

```
Enter an amount, for example, 11.56: 11.56 ↵ Enter
Your amount 11.56 consists of
    11 dollars
     2 quarters
     0 dimes
     1 nickels
     1 pennies
```

- Is it clear what the problem is asking of us?
 - ♦ Make sure you understand the question before starting



Program 8: Money Units

Step 1: Problem-solving Phase

1. Get the total amount of money by asking the user to enter a double value

```
money = input.nextDouble(); //Example: $11.56
```

2. Convert this amount into cents (multiply by 100)

```
totalCents = (int)money*100; // Example: 11.56 * 100 = 1156
```

3. Get the total number of **dollars** by now dividing by 100. And get remaining cents by using `totalCents % 100`.

```
totalDollars = totalCents / 100; //Example: 1156 / 100 = 11
```

```
remainingCents = totalCents % 100; //Example: 1156 % 100 = 56
```

4. Get the total # of **quarters** by dividing `remainingCents` by 25. And then recalculate `remainingCents`.

```
totalQuarters = remainingCents / 25; //Example: 56 / 25 = 2
```

```
remainingCents = remainingCents % 25; //Example: 56 % 25 = 6
```



Program 8: Money Units

Step 1: Problem-solving Phase

5. Get the total # of **dimes** by dividing `remainingCents` by 10. And then recalculate `remainingCents`.

```
totalDimes = remainingCents / 10; //Example: 6 / 10 = 0  
remainingCents = remainingCents % 10; //Example: 6 % 10 = 6
```

6. Get the total # of nickels by dividing `remainingCents` by 5. And then recalculate `remainingCents`.

```
totalDimes = remainingCents / 5; //Example: 6 / 5 = 1  
remainingCents = remainingCents % 5; //Example: 6 % 5 = 1
```

7. The value stored in `remainingCents` is the number of pennies left over
8. Display the result!



Program 8: Money Units

♦ Step 2: Implementation

LISTING 2.10 ComputeChange.java

```
1  import java.util.Scanner;
2
3  public class ComputeChange {
4      public static void main(String[] args) {
5          // Create a Scanner
6          Scanner input = new Scanner(System.in);
7
8          // Receive the amount
9          System.out.print(
10             "Enter an amount in double, for example 11.56: ");
11         double amount = input.nextDouble();
12
13         int remainingAmount = (int)(amount * 100);
14
15         // Find the number of one dollars
16         int numberOfOneDollars = remainingAmount / 100;
17         remainingAmount = remainingAmount % 100;
18
19         // Find the number of quarters in the remaining amount
20         int numberOfQuarters = remainingAmount / 25;
```

Program 8: Money Units

♦ Step 2: Implementation

```
21     remainingAmount = remainingAmount % 25;
22
23     // Find the number of dimes in the remaining amount
24     int numberOfDimes = remainingAmount / 10;
25     remainingAmount = remainingAmount % 10;
26
27     // Find the number of nickels in the remaining amount
28     int numberOfNickels = remainingAmount / 5;
29     remainingAmount = remainingAmount % 5;
30
31     // Find the number of pennies in the remaining amount
32     int numberOfPennies = remainingAmount;
33
34     // Display results
35     System.out.println("Your amount " + amount + " consists of");
36     System.out.println("    " + numberOfOneDollars + " dollars");
37     System.out.println("    " + numberOfQuarters + " quarters");
38     System.out.println("    " + numberOfDimes + " dimes");
39     System.out.println("    " + numberOfNickels + " nickels");
40     System.out.println("    " + numberOfPennies + " pennies");
41 }
42 }
```



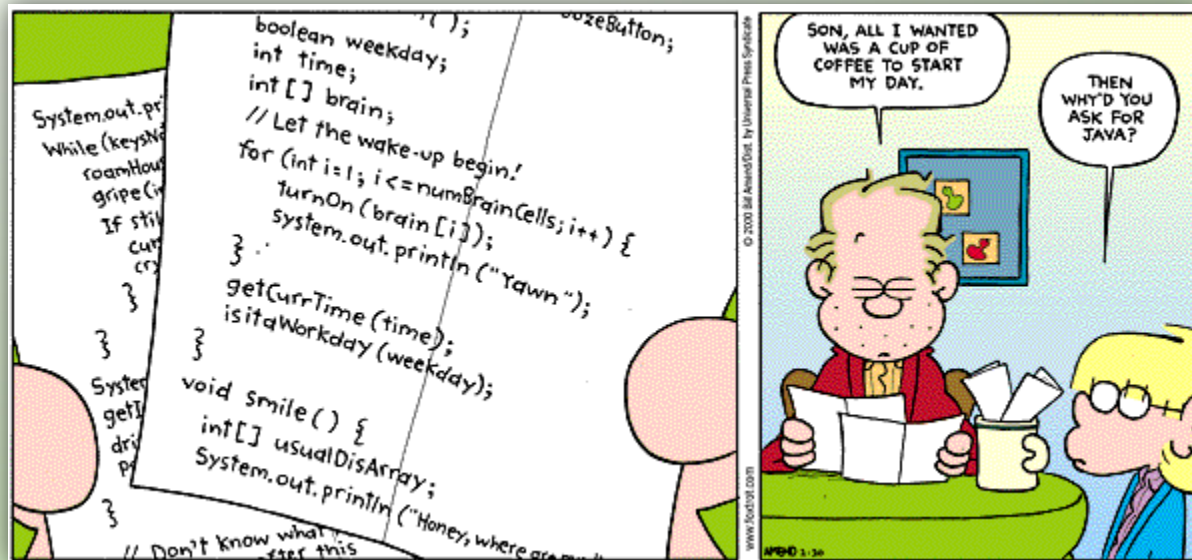
Program 8: Money Units

♦ Run the program:

```
Enter an amount, for example, 11.56: 11.56 ↵ Enter
Your amount 11.56 consists of
    11 dollars
     2 quarters
     0 dimes
     1 nickels
     1 pennies
```

♦ See sample program: `PrintMoney.java`

Characters and Strings in Java





Objectives

- ✦ To represent characters using the **char** type.
- ✦ To encode characters using ASCII and Unicode.
- ✦ To compare and test characters using the static methods in the **Character** class.
- ✦ To represent strings using the **String** objects.
- ✦ To return the string length using the **length()** method.
- ✦ To use the + operator to concatenate strings.
- ✦ To read strings from the console.
- ✦ To read a character from the console.
- ✦ To return a character in the string using the **charAt(i)** method (§4.4.2).
- ✦ To obtain substrings.
- ✦ To program using characters and strings (**GuessBirthday**).
- ✦ To format output using the **System.out.printf** method.



Motivations

This chapter introduced additional basic features of Java these features include Strings and Characters



Character Data Type

- ♦ Java allows you to process characters using the character data type, `char`
 - `char` represents a single character
 - a character literal is enclosed in single quotation marks
- ♦ Examples:
 - **`char letter = 'A';`**
 - ♦ Assigns the character `A` to the `char` variable `letter`
 - **`char numChar = '4';`**
 - ♦ Assigns the digit character `4` to the `char` variable `numChar`



Unicode and ASCII code

- ♦ Computers use binary numbers internally
 - a character is stored in a computer as a sequence of 0s and 1s
 - Mapping a character to its binary representation is called *encoding*
 - There are different ways to encode a character
 - Java uses **Unicode**
 - ♦ Unicode is an encoding scheme
 - ♦ Originally designed for 16-bit character encoding
 - ♦ This allowed for $2^{16} = 65,536$ characters



Unicode and ASCII code

♦ ASCII

- Most computers use ASCII
 - ♦ American Standard Code for International Exchange
- 8-bit encoding scheme
 - ♦ Used to represent all uppercase and lowercase letters, all digits, all punctuation marks, and control characters
 - ♦ 128 characters total
- Unicode includes ASCII code
 - ♦ `\u0000` to `\u007F` representing the 128 ASCII characters



Unicode and ASCII code

◆ ASCII

TABLE 4.4 ASCII Code for Commonly Used Characters

<i>Characters</i>	<i>Code Value in Decimal</i>	<i>Unicode Value</i>
'0' to '9'	48 to 57	\u0030 to \u0039
'A' to 'Z'	65 to 90	\u0041 to \u005A
'a' to 'z'	97 to 122	\u0061 to \u007A



Check Point

♦ What is the output of the following program:

```
public class Test {  
    public static void main(String[] args) {  
        char x = 'a';  
        char y = 'c';  
        System.out.println(++x);  
        System.out.println(y++);  
    }  
}
```

Output :

b

c



The String Type

- ♦ A **char** represents one character
 - ♦ So how do we represent a sequence (a string) of characters?
 - ♦ Use the data type called **String**
 - ♦ Example:
 - The following code declares variable **message** to be a **string** with the value "Welcome to Java"
- ```
String message = "Welcome to Java";
```



# The String Type

```
String message = "Welcome to Java";
```

- ◆ Here, **message** is a reference variable that references a string object with the contents "**Welcome to Java**"



# The String Type

## ♦ **String** details:

- Declaring a **String** variable:

```
String firstName;
```

- Assign a value to the **String** variable:

```
firstName = "Muhammad Alzahrani";
```



# Simple Methods for String Objects

## ♦ Getting String Length

- Use the **length()** method to return the number of characters in a string
- Example:

```
String message = "Welcome to Java";
System.out.println("The length of " +
 message + " is " + message.length());
```

### ♦ Output:

The length of Welcome to Java is 15



# Simple Methods for **String** Objects

## ♦ Concatenating Strings

- Remember: you can concatenate a number with a string
- At least one of the operands must be a string
- Examples:

```
String message = "Welcome " + "to " + "Java";
```

```
String s = "Chapter " + 2;
```

- s becomes "Chapter 2"

```
String s1 = "Supplement " + 'B';
```

- s1 becomes "Supplement B"



# Simple Methods for String Objects

## ♦ Concatenating Strings

- The augmented += operator can also be used for concatenation with strings
- Example:

```
String message = "Welcome to Java";
message += ", and Java is fun.";
System.out.println(message);
```

♦ Output:

♦ "Welcome to Java, and Java is fun."



# Simple Methods for String Objects

## ◆ Concatenating Strings

– Final example:

◆ If **i=1** and **j=2**, what is the output of the following:

◆ **System.out.println("i + j is " + i + j);**

◆ Output:

"i + j is 12"

◆ Why?

- In Java, we read from left to right
- So we have the String "**i + j is** " concatenated with the **int i**
- The result: a new **String** ("**i + j is 1**")
- This new **String** is concatenated with the **int j**
- You can force addition by enclosing the **i + j** in parentheses



# Reading a String from the Console

## ◆ How to read a string from the console

- Use the **next()** method on a Scanner object

```
System.out.print("Enter three words separated by spaces: ");
String s1 = input.next(); // assume we made Scanner object
String s2 = input.next();
String s3 = input.next();
System.out.println("s1 is " + s1);
System.out.println("s2 is " + s2);
System.out.println("s3 is " + s3);
```

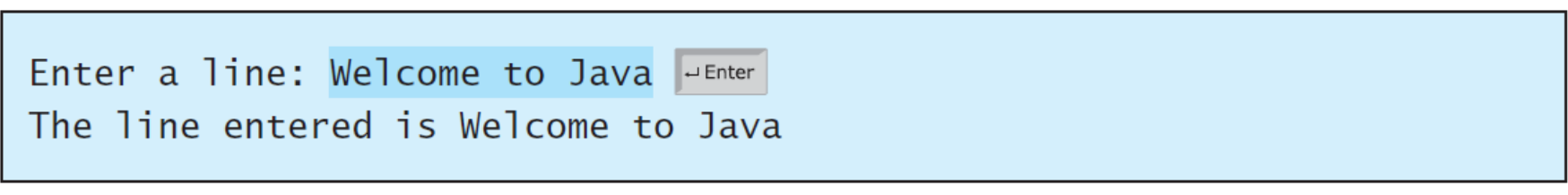
```
Enter three words separated by spaces: Welcome to Java 
s1 is Welcome
s2 is to
s3 is Java
```



# Reading a String from the Console

- ◆ How to read a complete line from the console
  - Use the **nextLine()** method on a Scanner object

```
Scanner input = new Scanner(System.in);
System.out.println("Enter a line: ");
String s = input.nextLine();
System.out.println("The line entered is " + s);
```



Enter a line: Welcome to Java ↵ Enter  
The line entered is Welcome to Java



# Check Point

♦ Show the output of the following expressions:

- a) `System.out.println("1" + 1);` **11**
- b) `System.out.println('1' + 1);` **50**
- c) `System.out.println("1" + 1 + 1);` **111**
- d) `System.out.println("1" + (1 + 1));` **12**
- e) `System.out.println('1' + 1 + 1);` **51**



# Simple Methods for String Objects

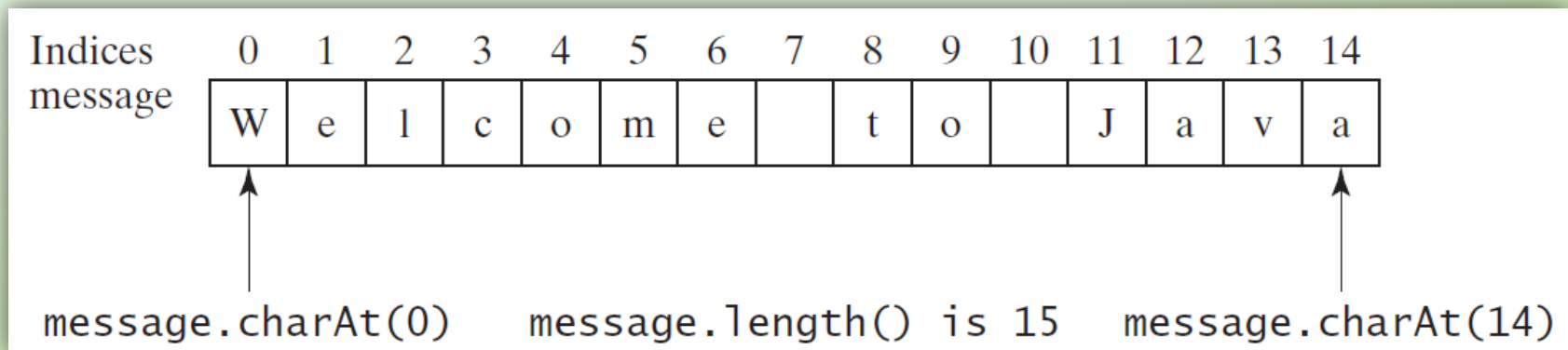
## ◆ Getting Characters from a String

– The `s.charAt(index)` method can be used to retrieve a specific character in a string `s`

◆ The index is between `0` and `s.length()-1`

◆ Example:

`message.charAt(0);` // Returns the character `W`





# Reading a Character from the Console

- ◆ How to read a single character from the console
  - Use the `nextLine()` method to read a string and then invoke the `charAt(0)` method on the string

```
Scanner input = new Scanner(System.in);
System.out.print("Enter a character: ");
String s = input.nextLine();
char ch = s.charAt(0);
System.out.println("The character entered is " + ch);
```



# Common Errors and Pitfalls

## ♦ Common Error # 1: Undeclared/Uninitialized Variables and Unused Variables

- a variable must be declared and a value assigned to it before you use it.
- Common errors include not declaring or not initializing a variable
- Example:

```
double interestRate = 0.05;
```

```
double interest = interestrates * 45;
```

- Java is case sensitive. In the 2<sup>nd</sup> line, interestrates is not defined, because the R is not capitalized



# Common Errors and Pitfalls

## ♦ Common Error #2: Integer Overflow

- Remember: numbers are stored with a limited number of digits
- If you try to save a value that is too large for a variable, an overflow error will occur
- Example:
  - ♦ `int value = 2147483647; // allowed`
    - but this number is the biggest possible int, therefore...
  - ♦ `value++; // will result in an error`



# Common Errors and Pitfalls

## ♦ Common Error #3: Round-off Errors

- a round-off error, also called a rounding error, is the difference between the exact mathematical value of a number and the approximated value that was calculated.
- Example:
  - ♦  $1/3$  is approximately 0.333 if you use three decimals
  - ♦ and it is 0.3333333 if you use seven decimal places
  - ♦ The number of digits you can store is limited
  - ♦ So having round-off errors is inevitable (a guarantee!)



# Common Errors and Pitfalls

## ♦ Common Error #3: Round-off Errors

- Calculations with floating-point numbers are also approximated

- ♦ Because they are not stored with complete accuracy

- Example:

- ♦ `System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);`

- The output should be 0.5, but the output is really  
0.50000000000000001

- ♦ `System.out.println(1.0 - 0.9);`

- The output should be 0.1, but the output is really  
0.099999999999999998

- Message: use integers for exact/precise results!



# Common Errors and Pitfalls

- ♦ Common Error #4: Unintended Integer Division
  - Perhaps you want “normal” division
    - ♦ Example:  $3 / 2 = 1.5$
  - However, if both operands are integers, Java will automatically use integer division
  - If you do not want integer division, you should make one of the operands a floating-point number

```
int number1 = 1;
int number2 = 2;
double average = (number1 + number2) / 2;
System.out.println(average);
```

(a)

```
int number1 = 1;
int number2 = 2;
double average = (number1 + number2) / 2.0;
System.out.println(average);
```

(b)



# Common Errors and Pitfalls

- ♦ Common Pitfall #1: Repeated Input Objects
  - New programmers often make many Scanner objects each time they want input
    - ♦ This is a mistake!
  - See the code below:

```
Scanner input = new Scanner(System.in);
System.out.print("Enter an integer: ");
int v1 = input.nextInt();
```

```
Scanner input1 = new Scanner(System.in);
System.out.print("Enter a double value: ");
double v2 = input1.nextDouble();
```

**BAD CODE**



# Common Errors and Pitfalls

- ♦ Common Pitfall #1: Repeated Input Objects
  - The code is not wrong, but it is inefficient (slow)
  - It creates two input objects, which is a waste
  - The correct code is below:

```
Scanner input = new Scanner(System.in);
System.out.print("Enter an integer: ");
int v1 = input.nextInt();
System.out.print("Enter a double value: ");
double v2 = input.nextDouble();
```

**GOOD CODE**